

Name:

Arooj Fatima

Internship ID:

ZYNVEX-CERT-1697

Type:

Project

Week:

Week 04

Topic:

Phishing Emails

ZYNVEX SOLUTIONS INTERNSHIP

PhishGuard: Phishing Email Detection System

GitHub repository: https://github.com/aroojfatimah399/PhishGuard
Live application: https://phishguard-aroojfatimah.streamlit.app/

1. Project Overview:

Phishing is one of the most common cyber-attack methods. Attackers send emails that imitate trusted organizations such as banks, delivery companies and online services in order to steal passwords, card details or money. Because these emails are written to look convincing, even careful users can be fooled, and manual filtering does not scale.
PhishGuard is a web application that analyses the text of an email and predicts whether it is legitimate or phishing. It does not only give a label: it produces a risk score and explains the decision by listing the suspicious indicators it found and highlighting the exact evidence in the email. The system combines two machine-learning classifiers trained on a real labelled dataset with a set of rule-based detectors for known phishing tactics.

2. Objectives:

1. Train and compare machine-learning models that classify emails as phishing or legitimate.
2. Make the results explainable, so a user can see why an email was flagged.
3. Evaluate the models rigorously using a held-out test set and cross-validation, while avoiding data leakage.
4. Deliver an interactive web dashboard that is deployed online and usable without installation.
5. Provide a documented, tested and version-controlled codebase that others can reproduce.

3. Features and Technologies Used:

3.1 Features:

- **Single email analysis:** paste a subject and body to receive a verdict, an ML probability, a rule-based score and a combined risk level (Low, Medium or High).
- **Explainable results:** indicator cards describe each warning sign, and the email text is displayed with the suspicious evidence highlighted by severity.
- **Influential words:** the words that pushed the model toward the phishing class are listed.
- **Two models compared:** TF-IDF with Logistic Regression and TF-IDF with Multinomial Naive Bayes.
- **Batch scanning:** upload a CSV of emails, scan all of them and download the results. If the file includes labels, accuracy is calculated.
- **Performance dashboard:** interactive charts and tables for accuracy, precision, recall, F1-score, ROC-AUC and confusion matrices.
- **Report export:** download an analysis as a PDF (links are defanged for safety) or JSON file.
- **Session history:** every scan in the current session is listed with its risk score.

3.2 Technologies:

Technology	Purpose
Python 3.12	Main programming language
scikit-learn	TF-IDF vectorisation, Logistic Regression, Naive Bayes, evaluation
pandas	Dataset loading and cleaning
Streamlit	Interactive web interface

Technology	Purpose
Altair, matplotlib, seaborn	Charts and confusion matrices
joblib	Saving and loading the trained models
fpdf2	PDF report generation
pytest	Automated testing
Git and GitHub	Version control and source hosting
Streamlit Community Cloud	Free public deployment

4. System Workflow and Architecture:

The system has two phases.

4.1 Training phase (offline):

1. **Dataset:** a labelled collection of phishing and legitimate emails.
2. **Preprocessing:** labels are normalised to 1 (phishing) and 0 (legitimate), then empty and duplicate emails are removed.
3. **Split:** a stratified 80/20 split creates a training set and a hold-out test set, preserving the class balance in both.
4. **Feature extraction and models:** each model is a scikit-learn Pipeline made of a TF-IDF vectoriser (unigrams and bigrams, English stop words removed) followed by a classifier. Keeping TF-IDF inside the pipeline means it is refitted on the training folds only during cross-validation, which prevents data leakage.
5. **Evaluation:** 5-fold stratified cross-validation on the training set, then a single evaluation on the test set.
6. **Model selection:** the best model is chosen by cross-validated F1-score, never by test-set score, so the test set stays an unbiased estimate. Models and metrics are saved with joblib and JSON.

4.2 Analysis phase (online):

1. The user submits an email through the Streamlit dashboard.
2. The selected model returns the probability that the email is phishing.
3. Rule-based detectors scan the text for urgency, credential requests, threats of suspension, prize or refund lures, payment and secrecy requests, generic greetings, pushy calls to click, attachment prompts and suspicious links. Link checks cover raw IP addresses, URL shorteners, look-alike brand domains, high-abuse top-level domains, missing HTTPS, excessive hyphens or subdomains, and punycode.
4. The two results are combined into one risk score:

Risk score = 0.75 x ML probability + 0.25 x rule score

The rule score is the sum of the severities of the distinct indicators found, divided by 10 and capped at 1. A risk below 0.35 is Low, below 0.65 is Medium and anything higher is High. The verdict (likely phishing or likely legitimate) follows the ML probability at a 0.5 threshold.

5. The dashboard shows the gauge, indicator cards and highlighted evidence, and offers PDF and JSON export.

The highlighted text is HTML-escaped before display, and links are never clickable, so analysing a malicious email cannot harm the user.

5. Installation and Setup:

Requirements: Python 3.10 or newer and Git. The commands below are for Windows PowerShell.

1. Clone the repository and open it in VS Code:

```
git clone https://github.com/aroojfatimah399/PhishGuard.git cd
PhishGuard
```

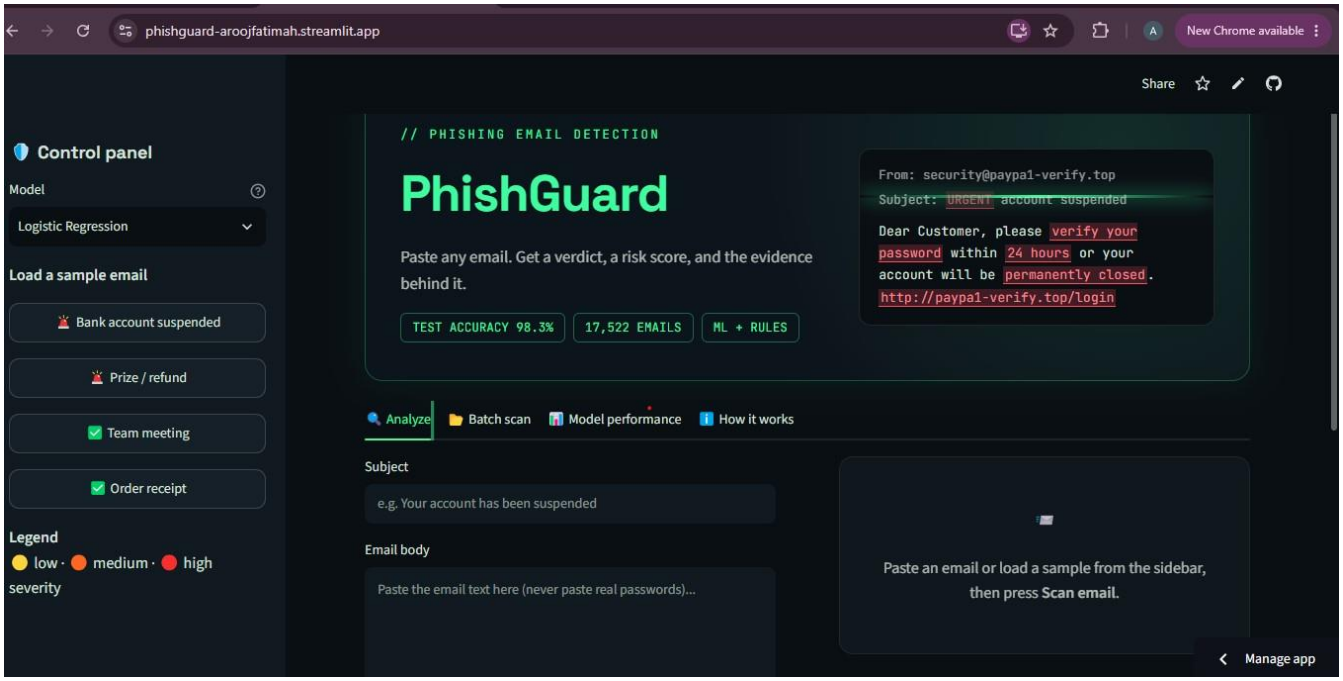
2. Create and activate a virtual environment:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
```

3. Install the dependencies:

```
python -m pip install --upgrade pip pip
install -r requirements.txt
```

4. Prepare the dataset (optional, to retrain). Download a phishing email dataset to data/raw/, save it as



CSV, and convert it:

```
python prepare_dataset.py data/raw/Phishing_Email.csv
```

5. Train and evaluate the models (optional, a trained model is included):

```
python train.py
```

6. Run the tests:

```
python -m pytest -q
```

7. Start the application, which opens at http://localhost:8501:

```
streamlit run app.py
```

The application is also deployed online at [PASTE YOUR STREAMLIT URL], so no installation is needed to try it.

6. Usage and Screenshots:

Analyzing an email: Paste a subject and body into the Analyze tab, or load one of the four sample emails from the sidebar, then press Scan email. A short scanning sequence runs, and the result shows an animated risk gauge, the verdict, the ML probability, the rule score and the number of indicators. Below it, indicator cards explain each warning sign and the email is shown with the evidence highlighted in yellow, orange or red by severity.

FIGURE: home page with hero and live URL visible

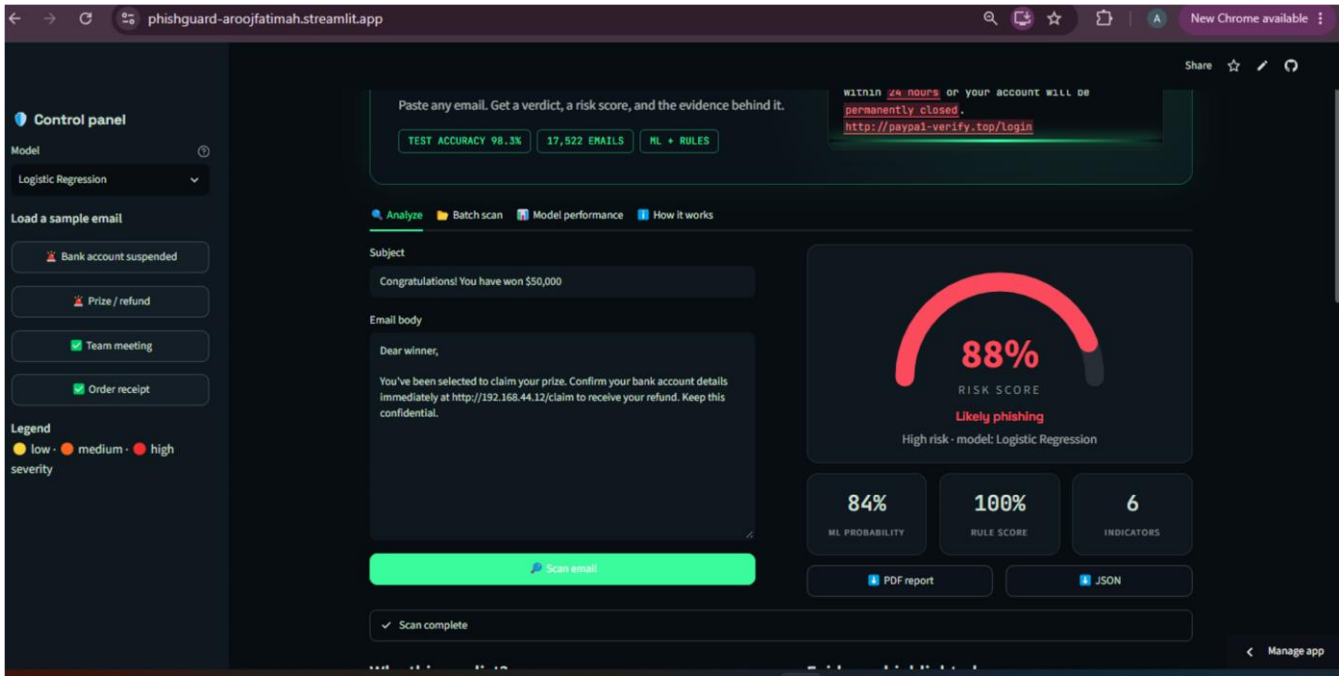


FIGURE: phishing example showing High risk

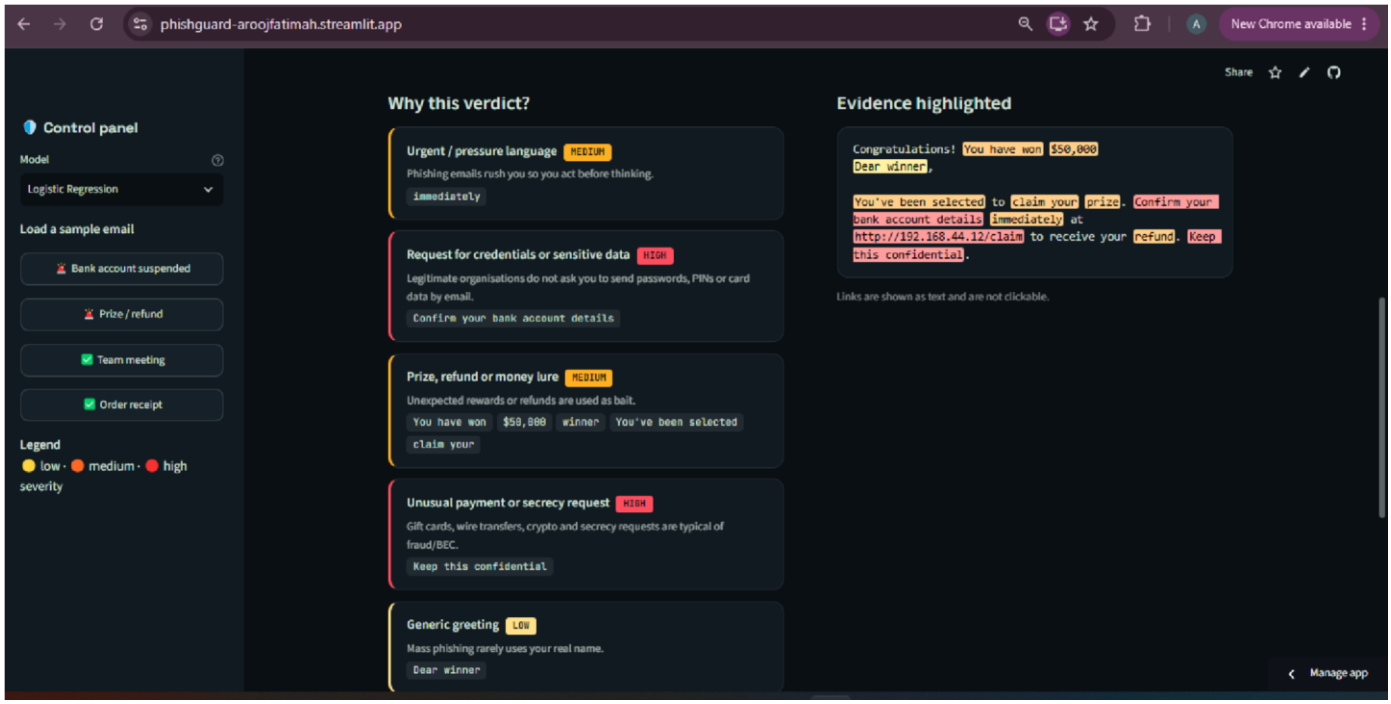
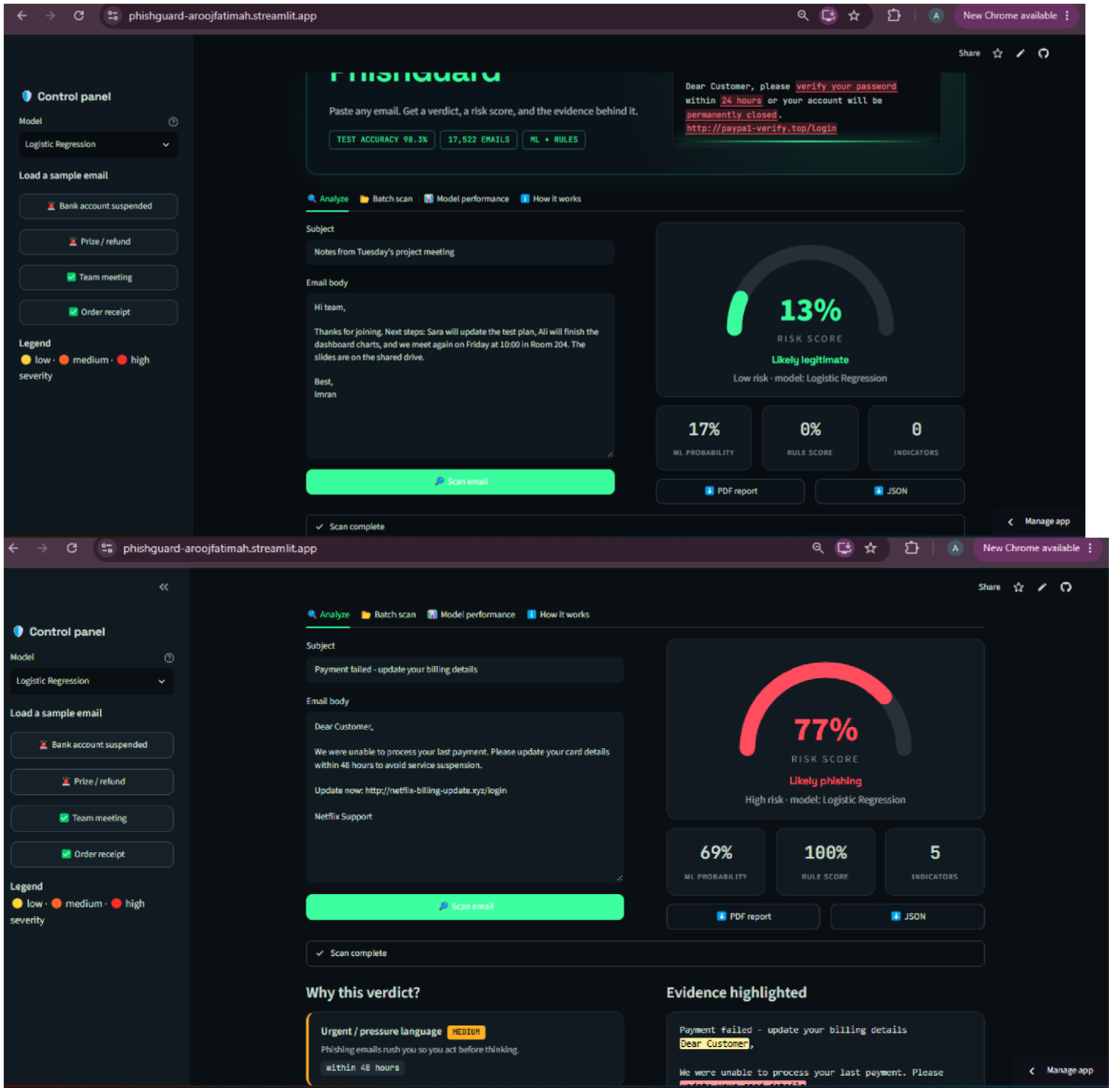


FIGURE: indicator cards and highlighted evidence FIGURE: legitimate



example showing Low risk

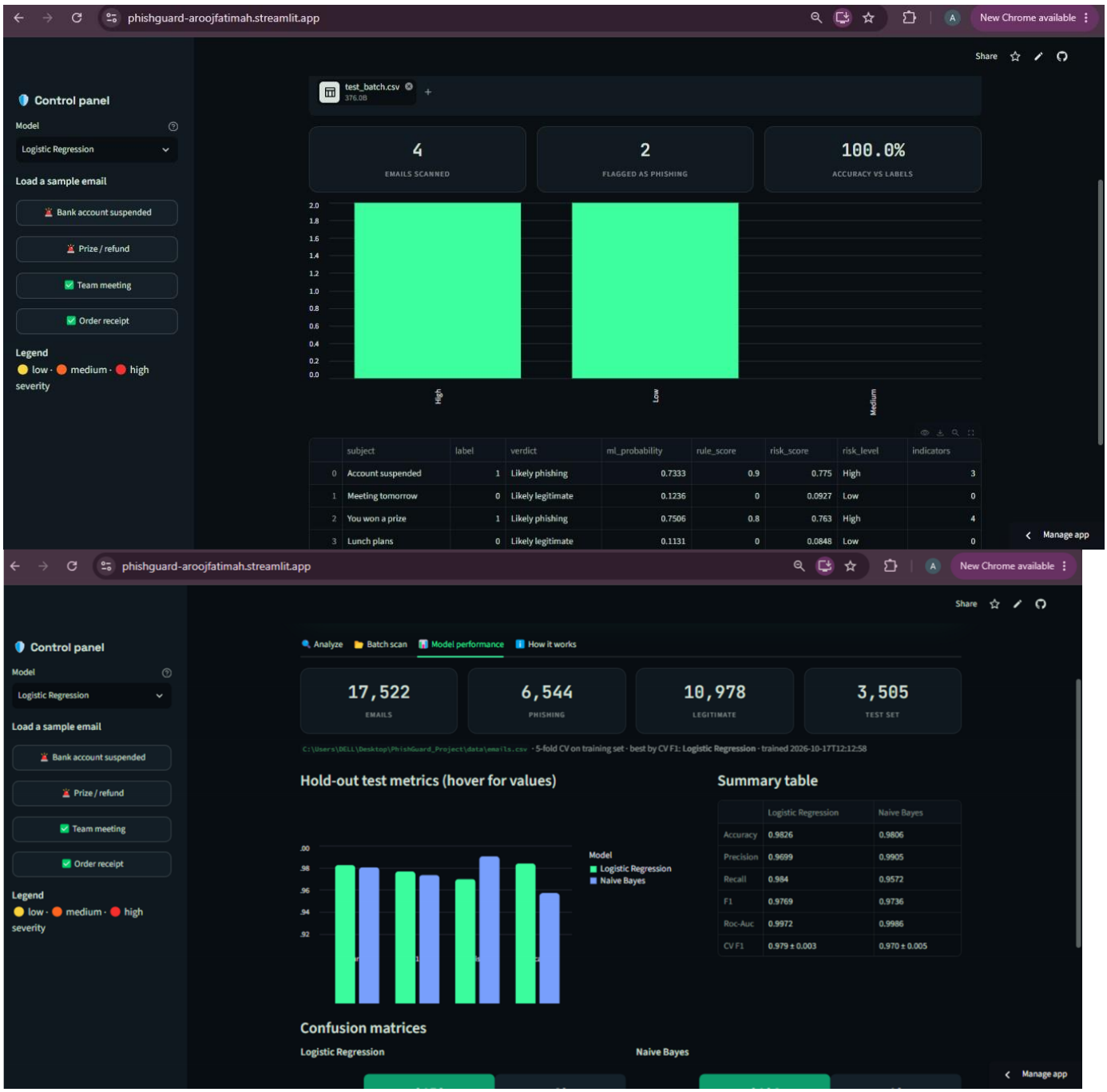


FIGURE: custom email analysed (billing-update phishing example)

Batch scanning: The Batch scan tab accepts a CSV with subject and body columns (and an optional label column). It reports how many emails were scanned and flagged, shows a risk-level chart and offers the results as a download.

FIGURE: batch scan results

Model performance: The Model performance tab presents the dataset size, an interactive metric comparison, a summary table and the confusion matrices of both models.

FIGURE: model performance chart and table

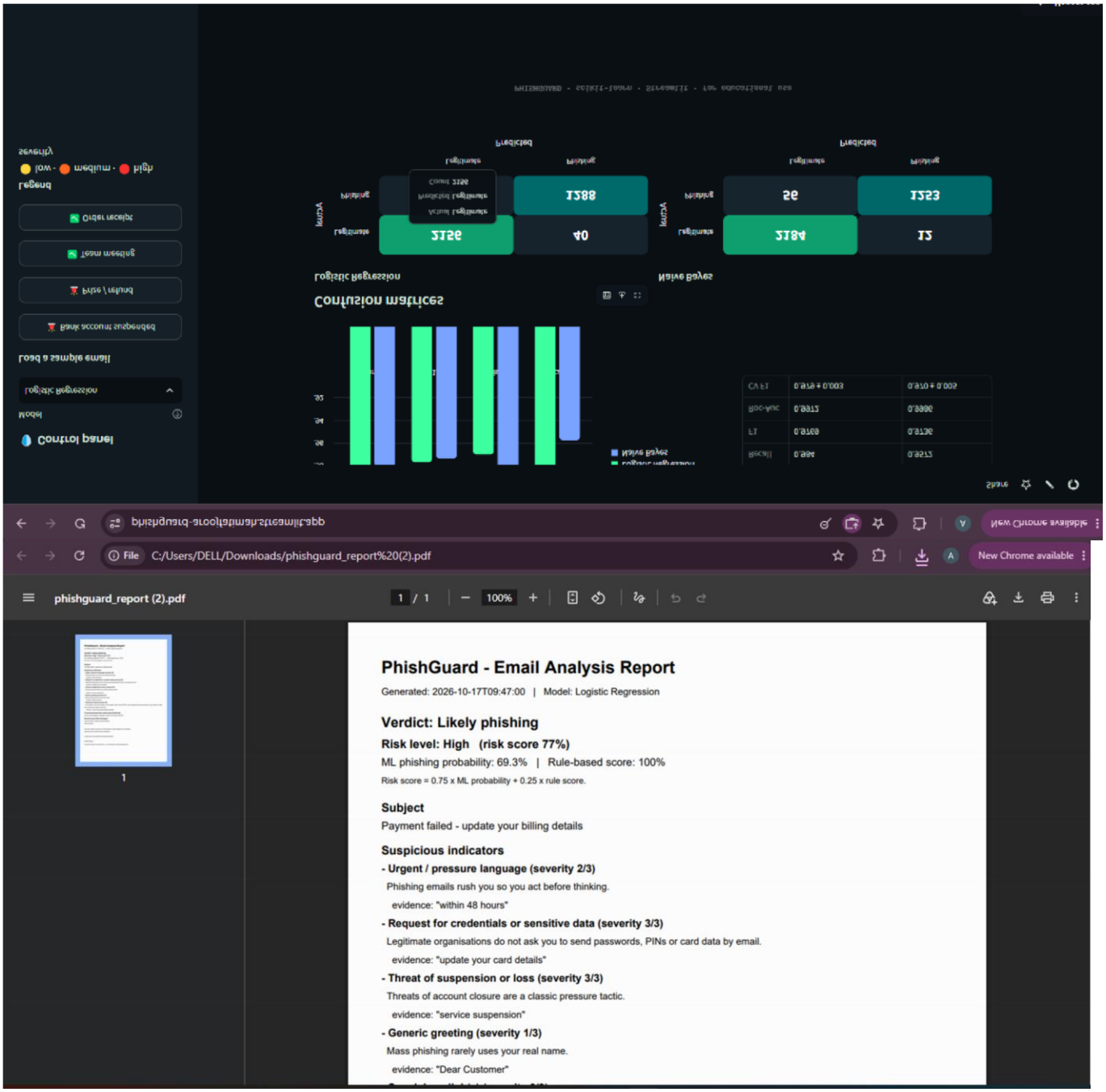


FIGURE: confusion matrices

PDF report: The PDF export contains the verdict, scores, indicators with evidence, influential words and an excerpt of the email with links defanged.

FIGURE: exported PDF report

7. Dataset:

The models were trained on a public phishing email dataset downloaded from Kaggle: [Phishing Email Detection], by [Cyber Cop], available at [https://www.kaggle.com/datasets/subhajournal/phishingemails] . The raw file contained 18,650 rows. After converting the labels to a binary scheme and removing empty rows and duplicates, 17,536 emails remained, and a further 14 exact duplicates were removed when the data was loaded for training, leaving **17,522 emails**.

Class	Emails	Share
Legitimate (0)	10,978	62.7%
Phishing (1)	6,544	37.3%
Total	**17,522**	100%

The data was split with stratification into **14,017 training emails** and **3,505 test emails**. Because the classes are imbalanced, the Logistic Regression model uses balanced class weights and results are reported with precision, recall and F1-score in addition to accuracy.
The repository also contains data/sample_emails.csv, a small synthetic dataset generated by tools/make_sample_data.py. It is used only for demonstrations and unit tests, and no reported result comes from it.

8. Testing and Results:

8.1 Evaluation method:

The training set was evaluated with 5-fold stratified cross-validation. The final models were then evaluated once on the untouched test set of 3,505 emails. The best model was selected by cross-validated F1-score. In phishing detection, recall matters particularly, since each missed phishing email is a potential victim, while precision measures how many alerts are genuine.

8.2 Results:

Metric	Logistic Regression	Naive Bayes
Accuracy	0.983	0.981
Precision	0.970	0.991
Recall	0.984	0.957
F1-score	0.977	0.974
ROC-AUC	0.997	0.999
Cross-validated F1 (mean ± std)	0.979 ± 0.003	0.970 ± 0.005

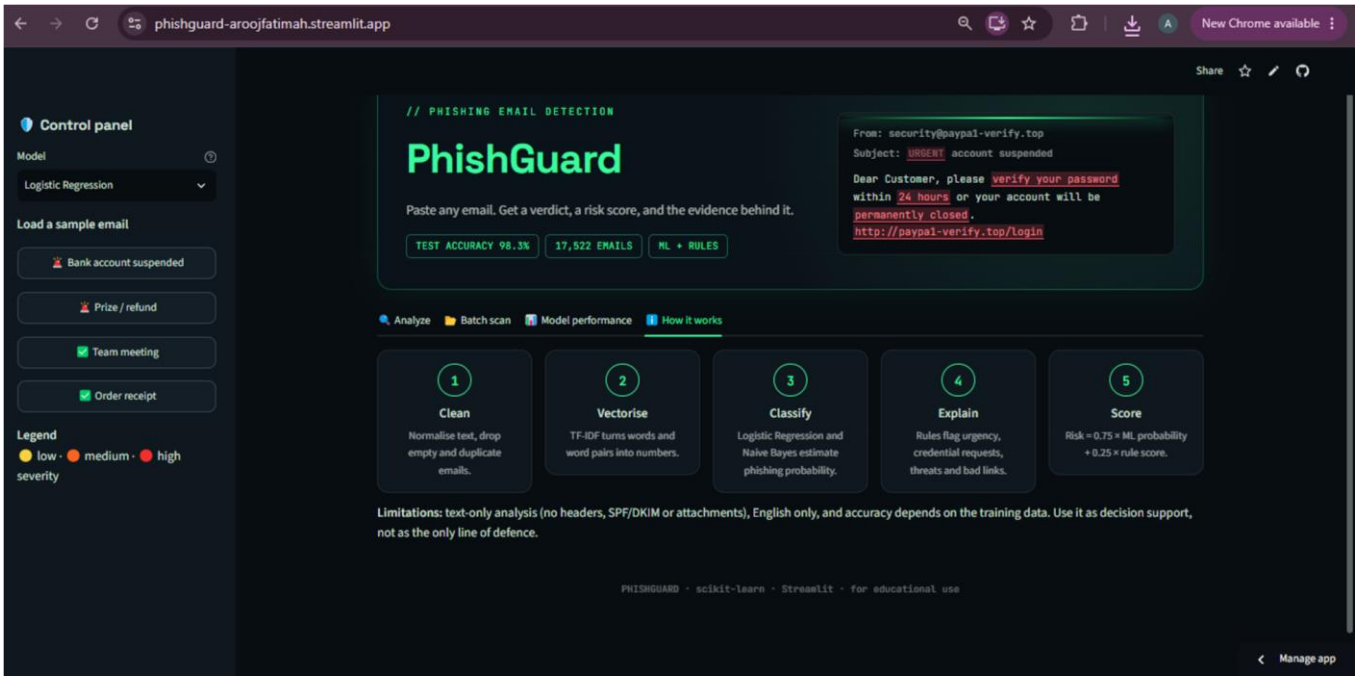


FIGURE: confusion matrices, with the TN, FP, FN and TP counts read from the Model performance tab

8.3 Discussion:

Both models perform strongly and similarly. **Logistic Regression** was selected as the default model because it had the higher cross-validated F1-score (0.979 against 0.970). It has the higher recall (0.984), so it misses fewer phishing emails, at the cost of slightly lower precision (0.970), meaning a few more legitimate emails are flagged. **Naive Bayes** is the opposite: its precision of 0.991 gives very few false alarms, but its recall of 0.957 lets more phishing emails through. For a security tool where a missed attack is usually costlier than a false alarm, the higher-recall model is the better default, and the dashboard lets the user switch between the two.

The low standard deviation across the cross-validation folds (0.003 and 0.005) shows that the results are stable and not the product of one lucky split.

8.4 Software testing

Ten automated pytest tests cover the indicator rules, URL analysis, risk scoring and levels, escaping of HTML in the highlighted output, dataset loading, model training and its metrics, confirmation that TF-IDF sits inside the pipeline (so no leakage occurs), the ordering of risk for phishing and legitimate examples, and PDF and JSON export. All 10 tests pass (`python -m pytest -q`). The deployed application was also tested manually with phishing and legitimate sample emails, a custom email, a batch CSV and PDF export.

9. Challenges:

1. **Dataset preparation.** The Kaggle dataset arrived as an Excel file with its own label names, and the pipeline expected a CSV with binary labels. A conversion script was written that detects columns, normalises labels and removes 1,114 empty or duplicate rows.
2. **Avoiding data leakage.** Fitting TF-IDF on all the data before splitting would leak test information into training and inflate the scores. Placing the vectoriser inside the pipeline solved this.
3. **Class imbalance.** Roughly 63% of emails are legitimate, so accuracy alone can mislead. Balanced class weights were used and precision, recall and F1 were reported.
4. **Balancing accuracy and explainability.** A pure ML score is hard to trust. Combining it with transparent rules and evidence highlighting made each decision understandable.
5. **Safe handling of malicious content.** Highlighted text is HTML-escaped, links are not clickable, and URLs in PDF reports are defanged.

6. **Version and deployment issues.** The saved model was trained with a different scikit-learn version than the one installed locally, which produced warnings. Retraining on the local machine and pinning the library version fixed it.
7. **Git and GitHub.** The first push failed because a different GitHub account was saved in Windows Credential Manager. A later push was rejected because the remote repository had a commit that the local one lacked, which was resolved with a rebase before pushing.

Limitations: The system analyses text only: it does not inspect email headers, SPF/DKIM results, sender reputation or attachments. It is trained on English text. The dataset appears to combine emails from several collections, so a model may partly learn the style of a source rather than true phishing characteristics, and accuracy on real-world mail may be lower than the figures above. It should be used as decision support, not as a sole line of defence.

10. Future Improvements:

- Parse raw .eml files to use headers, the Reply-To and From mismatch, and SPF/DKIM/DMARC results.
- Check links against URL reputation services and analyse attachments.
- Evaluate transformer models such as BERT or DistilBERT.
- Add multilingual support.
- Add SHAP or LIME explanations for deeper model transparency.
- Test on a second independent dataset to measure generalisation.
- Add user feedback so that wrongly classified emails can be used for retraining.
- Offer an email-client or browser extension, and an API.

11. Conclusion:

PhishGuard demonstrates that a lightweight, classical machine-learning approach can detect phishing emails with high accuracy and, just as importantly, explain its decisions. Trained on 17,522 real emails, the best model (TF-IDF with Logistic Regression) reached 98.3% accuracy, 97.0% precision, 98.4% recall and a 0.977 F1-score on an unseen test set, with stable cross-validation results. Combining the model with rulebased indicators, evidence highlighting and a clear risk score turns a black-box prediction into something a user can act on. The project is open source on GitHub, covered by automated tests, and deployed as a public web application.

12. References:

1. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
2. Streamlit Documentation. https://docs.streamlit.io
3. scikit-learn Documentation: TfidfVectorizer, LogisticRegression, MultinomialNB, cross_validate. https://scikit-learn.org
4. Anti-Phishing Working Group (APWG). Phishing Activity Trends Reports. https://apwg.org
5. [DATASET CITATION: author, title, Kaggle URL, access date 17 October 2026]
6. PhishGuard source code. https://github.com/aroojfatimah399/PhishGuard